



Instituto Tecnológico de Las Américas (ITLA)

Fundamentos de Criptografía.

Proyecto Final API-AseguradorasRD

Javier Cedano 2024-0768

Docente: Engel Lopez.

27 de marzo 2025

Contents

Paso 1: Crear la API en Node.js	3
Paso 2: Subir el Proyecto a GitHub	4
Paso 3: Desplegar en la Nube con Render	5
Paso 4: Probar la API en la Nube	5
✔ Verificar API: GET https://api-aseguradorasrd.onrender.com/ ..	6
✔ Obtener Aseguradoras:	6
✔ Iniciar sesión: POST https://api-aseguradorasrd.onrender.com/login	7
✔ Agregar Aseguradora:	9
✔ Actualizar Aseguradora: PUT https://api-aseguradorasrd.onrender.com/aseguradoras/ARS%20Cedano	10
✔ Eliminar Aseguradora: DEL https://api-aseguradorasrd.onrender.com/aseguradoras/ARS%20Cedano	11
✔ Obtener Usuarios: GET https://api-aseguradorasrd.onrender.com/users	12
✔ Agregar Usuario: POST https://api-aseguradorasrd.onrender.com/users	12
Conclusión	15

Paso 1: Crear la API en Node.js

Esta API desarrollada con Node.js y Express proporciona un sistema de gestión de usuarios con autenticación básica. Entre sus principales características se incluyen:

En `app.js`, importamos Express, creamos una instancia de la aplicación, y configuramos el manejo de datos en formato JSON. Definimos una lista de usuarios como base de datos temporal y creamos rutas básicas como la raíz (`/`), la obtención de usuarios (`/users`), la adición de nuevos usuarios, y la autenticación con un json de users que tiene credenciales predefinidas, al que también se le pueden agregar usuarios. Todo esto funcionando con seguridad mediante un middleware que cuenta con un JWT.

Finalmente, configuramos el servidor para que escuche en un puerto dinámico y lo iniciamos. También instalamos las dependencias necesarias, como Express, jsonwebtoken, fs, path, bcrypt, dotenv, entre otros.

Este código es una API en Node.js con Express que gestiona aseguradoras de salud en República Dominicana. Se utilizaron las siguientes librerías:

Librerías utilizadas

Librería	Función
express	Framework web para manejar rutas y peticiones HTTP.
jsonwebtoken (jwt)	Genera y valida tokens JWT para autenticación.
bcrypt	Cifra y compara contraseñas para mayor seguridad.
fs	Permite leer y escribir archivos (se usa para almacenar aseguradoras).
path	Maneja rutas de archivos en el sistema operativo.
dotenv	Carga variables de entorno desde un archivo <code>.env</code> .
os (<i>aunque solo se importa totalmem y no se usa</i>)	Permite obtener información del sistema operativo.

Flujo del código

1. Carga de datos

- Se lee un archivo llamado `aseguradoras.json` y se carga en memoria.
- Si hay un error al leerlo, la API se detiene.

2. Usuarios y autenticación

- Hay una base de datos simulada (`DBusers`) con contraseñas cifradas con `bcrypt`, estos se guardan en un json llamado `user.json`.
- Existe un endpoint `/login` donde el usuario se autentica y recibe un token JWT válido por 1 hora.
- Se usa un middleware de autenticación (`authMiddleware`) para proteger rutas privadas.

3. Rutas de la API

Públicas (cualquiera puede acceder):

- `/` → Página de bienvenida con documentación.

- /aseguradoras → Devuelve la lista de aseguradoras.
- /aseguradoras/:nombre → Busca una aseguradora específica por nombre.

Protegidas (requieren autenticación con un token JWT):

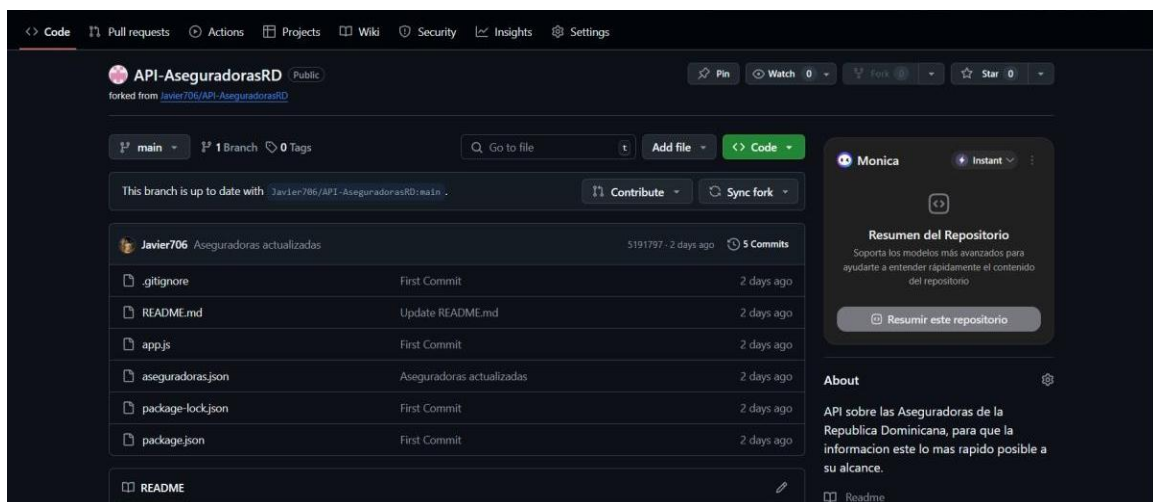
- POST /aseguradoras → Agrega una nueva aseguradora.
- PUT /aseguradoras/:nombre → Actualiza la información de una aseguradora.
- DELETE /aseguradoras/:nombre → Elimina una aseguradora.
- POST /users → Crear nuevo usuario.
- GET /users → Devuelve los nombres de los usuarios.

4. El servidor

La API corre en el puerto que se define en .env, y si no hay uno configurado, usa 3000 por defecto.

Nota: Al código ser tan extenso no lo agregamos a este documento, pero esta es una descripción detallada de lo que se implementó

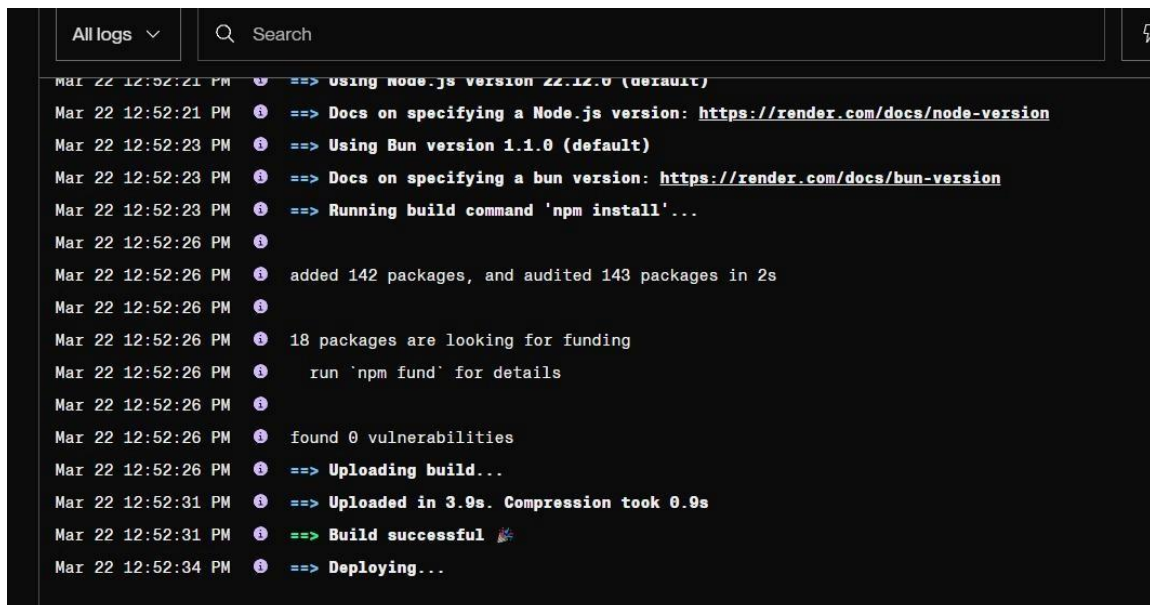
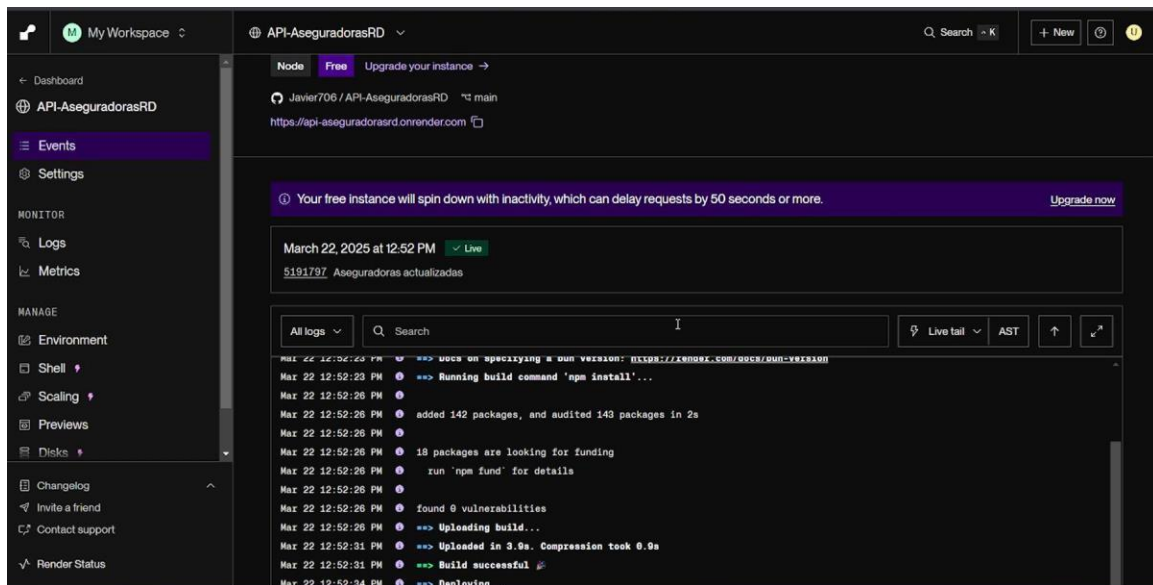
Paso 2: Subir el Proyecto a GitHub



En este proceso, mostramos cómo llevamos a cabo el despliegue de nuestra API. En cada uno de los repositorios involucrados, se encuentran las líneas de código específicas que hemos utilizado para configurar y ejecutar la API correctamente. Estos repositorios contienen no solo los archivos fuente de la API, sino también los scripts y configuraciones necesarios para su puesta en marcha. Cada commit o versión en estos repositorios refleja un paso en el proceso de despliegue, lo que facilita el seguimiento de las modificaciones realizadas y la evolución del sistema. Además, las configuraciones específicas, como las variables de entorno, las dependencias del sistema y los parámetros de red, se encuentran claramente definidos en los archivos correspondientes para asegurar que el despliegue se realice sin contratiempos.

A la hora de subir el proyecto a github se tomo medidas de seguridad como no subir el archivo .env donde teníamos la password del JWT, para luego definirla como variable de entorno en Render.

Paso 3: Desplegar en la Nube con Render



En este paso, hemos completado el despliegue de nuestra API en la plataforma de Render. Render es un servicio de infraestructura en la nube que permite a los desarrolladores desplegar aplicaciones web de manera rápida y sencilla. Tras configurar los repositorios y asegurarnos de que el código estuviera listo, procedimos a vincular el repositorio correspondiente con Render, lo que permitió que la plataforma gestionara automáticamente el proceso de despliegue, incluyendo la creación de instancias y la asignación de recursos necesarios.

Para desplegar la API en Render se usaron los siguientes parametros

Build Command: npm install

Start Command: node app.js

Variable de entorno: JWT_SECRET; valor: !As^&2r@d*r@\$!

Paso 4: Probar la API en la Nube

Una vez que ya subimos la API en la Nube comenzaremos a realizar las consultas.

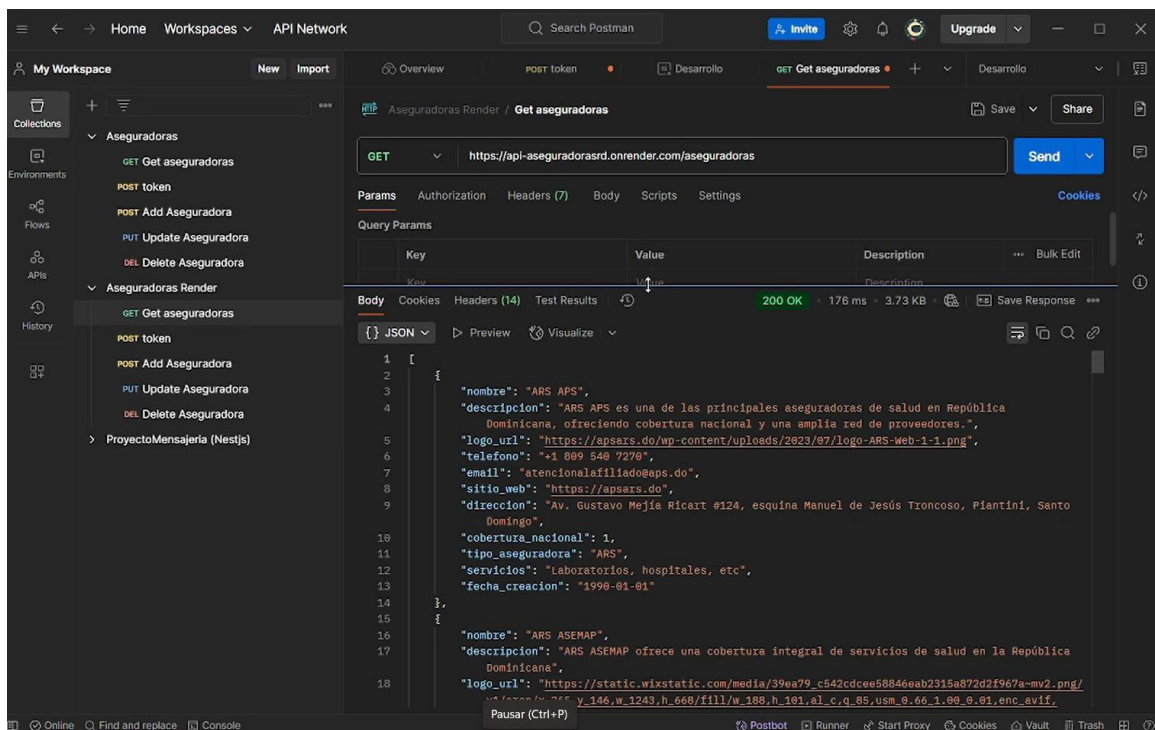
Cuando Render termine el despliegue, mostrará una **URL pública**:

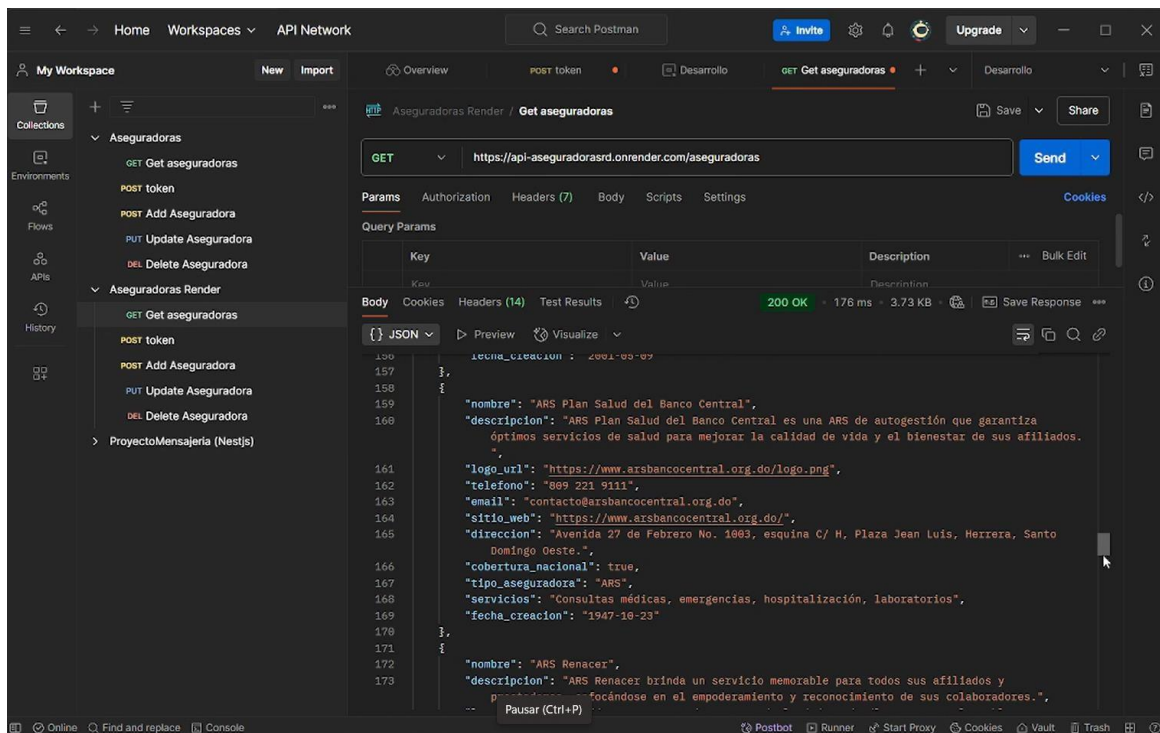
<https://api-aseguradorasrd.onrender.com>



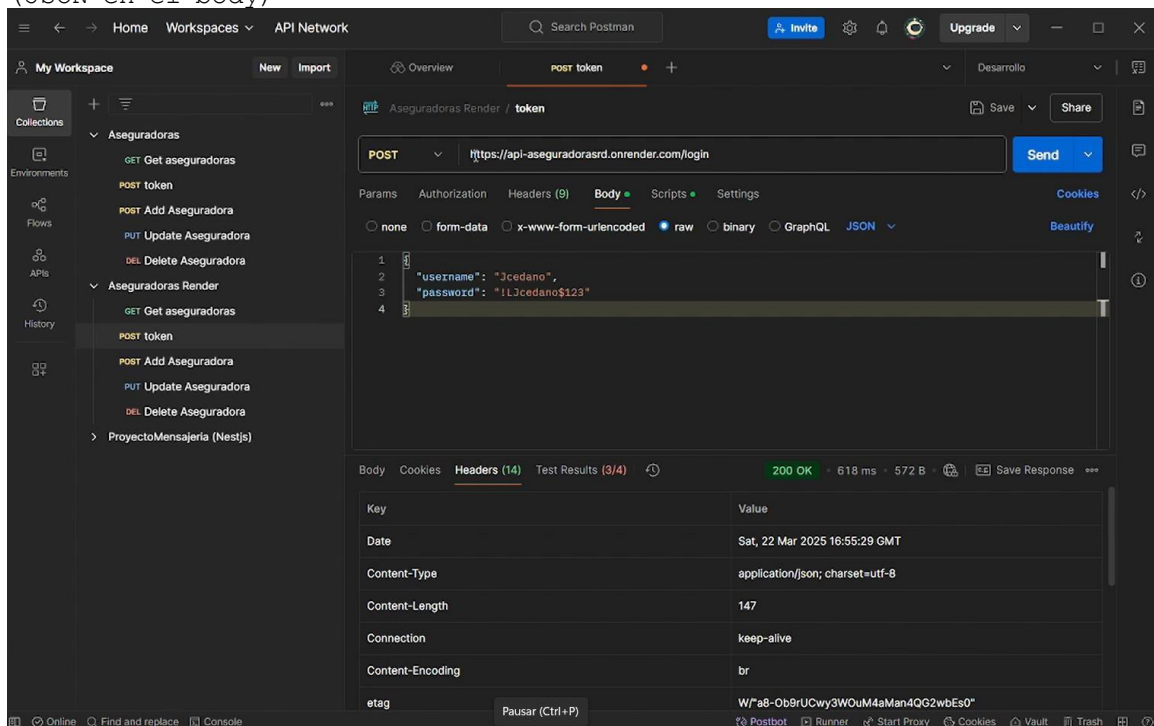
✓ **Verificar API: GET** <https://api-aseguradorasrd.onrender.com/>

✓ **Obtener Aseguradoras:**
GET <https://api-aseguradorasrd.onrender.com/aseguradoras>





✓ Iniciar sesi\u00f3n: POST <https://api-aseguradorasrd.onrender.com/login>
(JSON en el body)



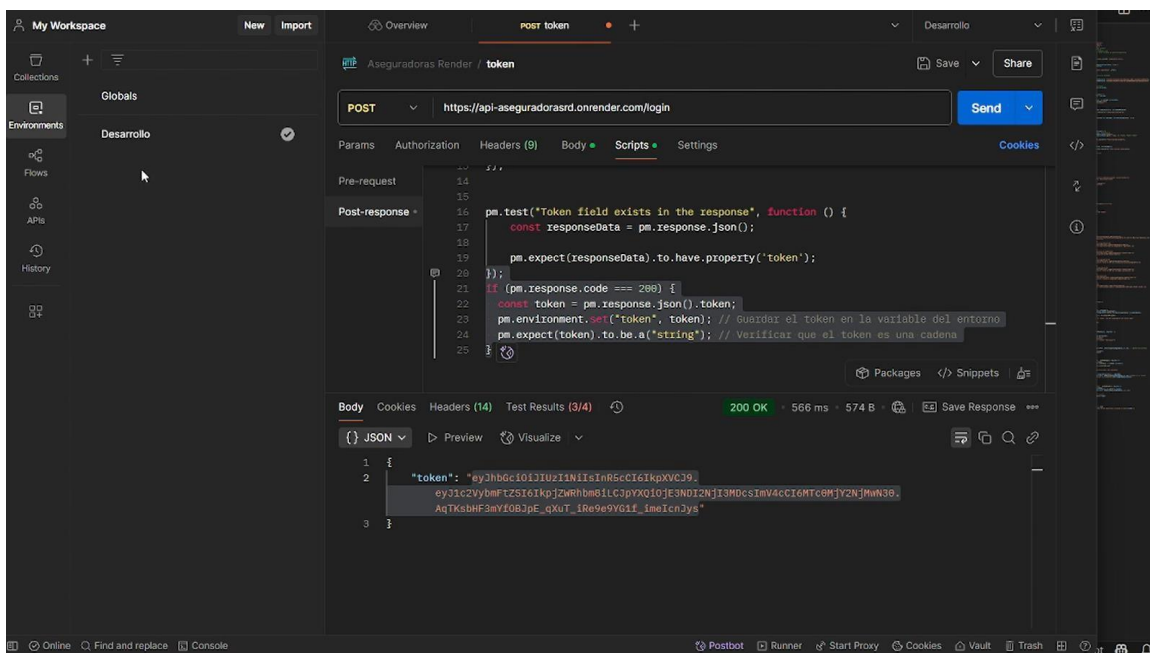

```

{} users.json > {} 0
1  [
2    {
3      "username": "Jcedano",
4      "hashedPassword": "$2b$10$LJnTZUIaHeSw77ZBr3NO9uhc8Cg.Wp0Kj.G43S2DAJT14hMj9DUfK"
5    },
6    {
7      "username": "GBadmin",
8      "hashedPassword": "$2b$10$kLsFU15PhCt7lo0AbrvVB.qopE988YpDh9cRLgSLHiEdapq4nS9S2"
9    },
10   {
11     "username": "GRadmin",
12     "hashedPassword": "$2b$10$OT1MAhUz3EiHwASLkizneU5mygUNqxr5oB00LmRV/uaTmdmVU0VS"
13   }
14 ]

```

Cuando se realiza una solicitud POST, el servidor valida las credenciales del usuario (generalmente un nombre de usuario y contraseña) estos usuarios anteriormente creados se guardaran en el json users.json al cual se hará un request y sus contraseñas que están con un hash se comparan con la contraseña que introduciremos, para eso se utiliza bcrypt.compare para saber si la contraseña es valida. Si la autenticación es exitosa, el servidor responde con un **token de acceso**, que suele ser un **JSON Web Token (JWT)**.

El token se utiliza para autenticar y autorizar futuras solicitudes a rutas protegidas de la API. En otras palabras, prueba que el usuario ha iniciado sesión y tiene permiso para acceder a ciertos recursos. También nos permite crear, modificar o eliminar registros de aseguradoras, pólizas, etc.



Una vez que ya tenemos nuestro token con los permisos necesarios podremos modificar las aseguradoras que tenemos.

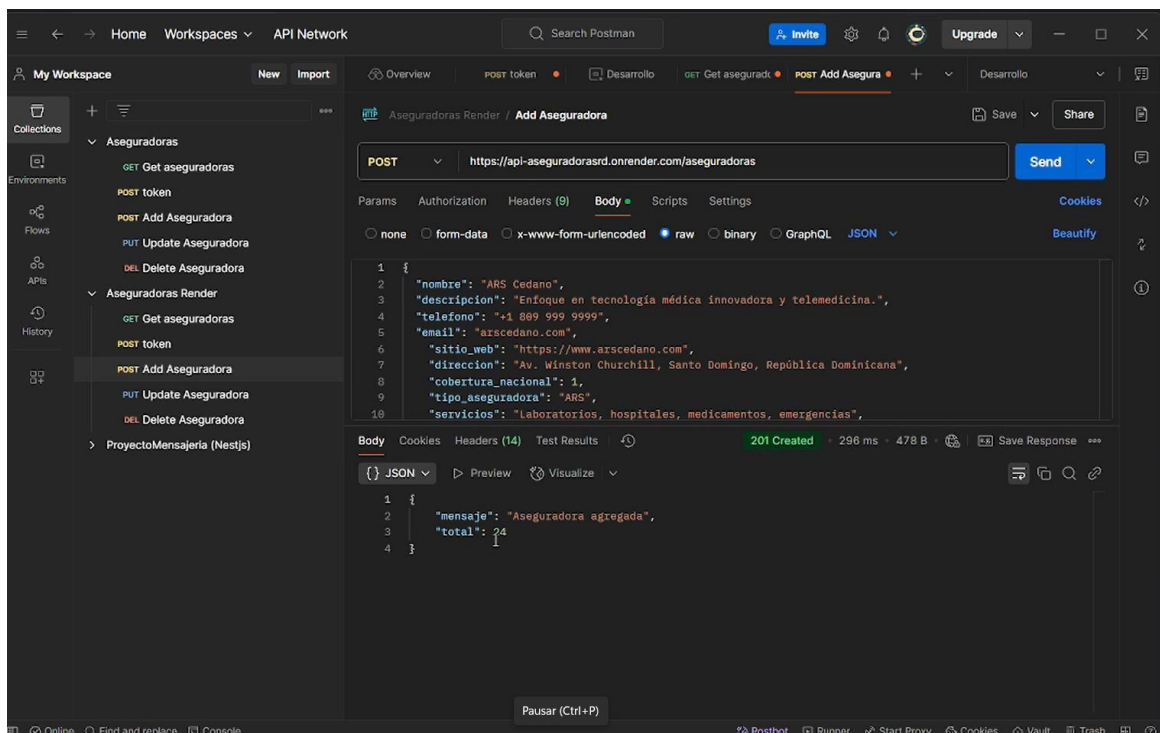
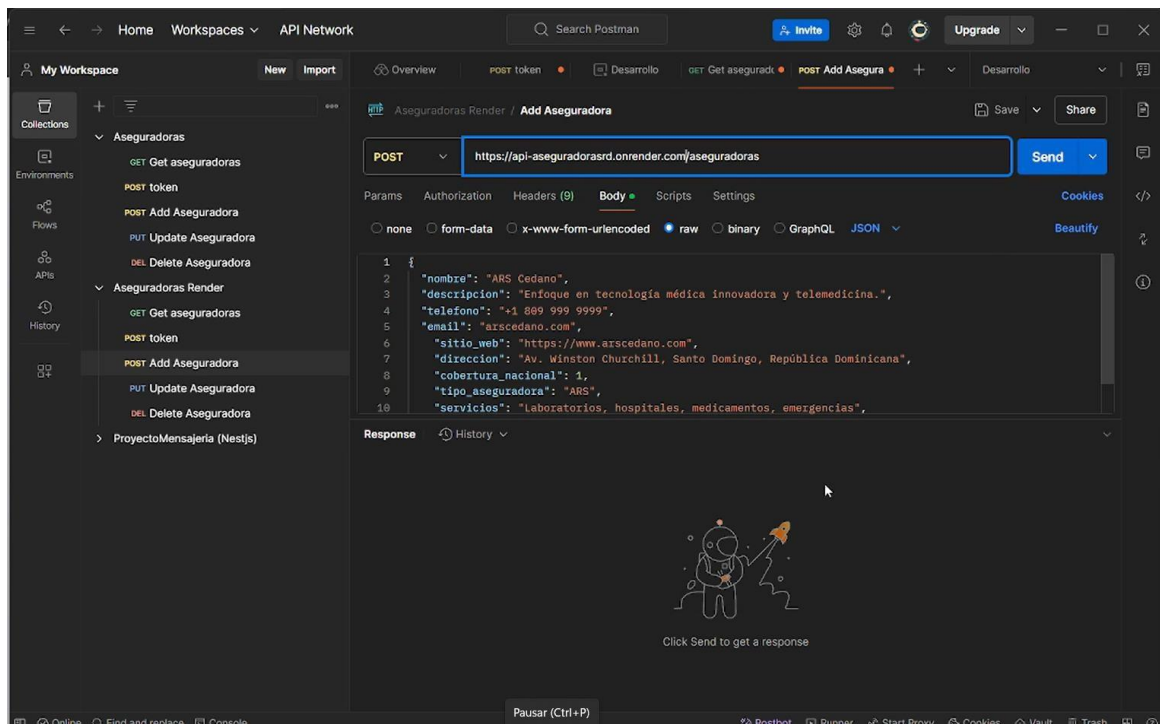
Teniendo en cuenta que para realizar estas acciones como actualizar, agregar o eliminar se precisa que el token este en el header en este caso se utilizo como variable de entorno en postman, con un script sencillo de modo que para referenciarlo en el header seria de la siguiente manera:

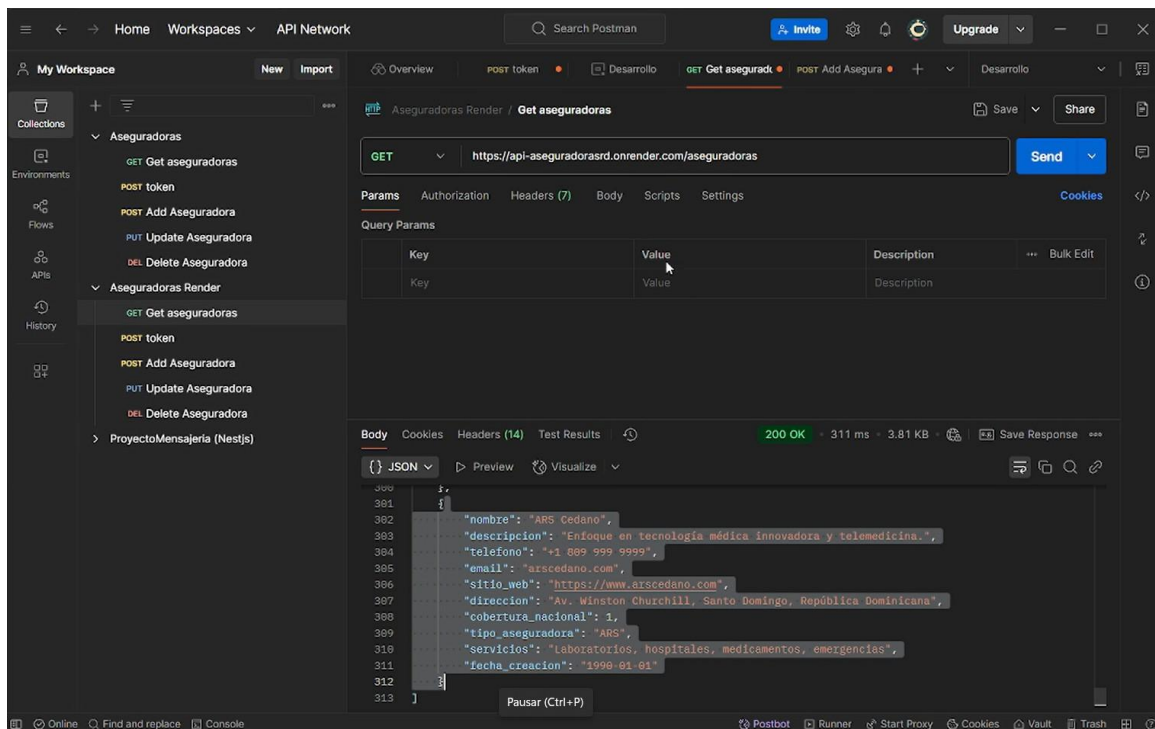
Headers		
	Key	Value
	Authorization	Bearer <code>{{token}}</code>

El value Bearer `{{token}}` esta haciendo referencia a la variable de entorno donde tenemos guardado el token, con esto se puede realizar las acciones que requieren el token.

✓ Agregar Aseguradora:

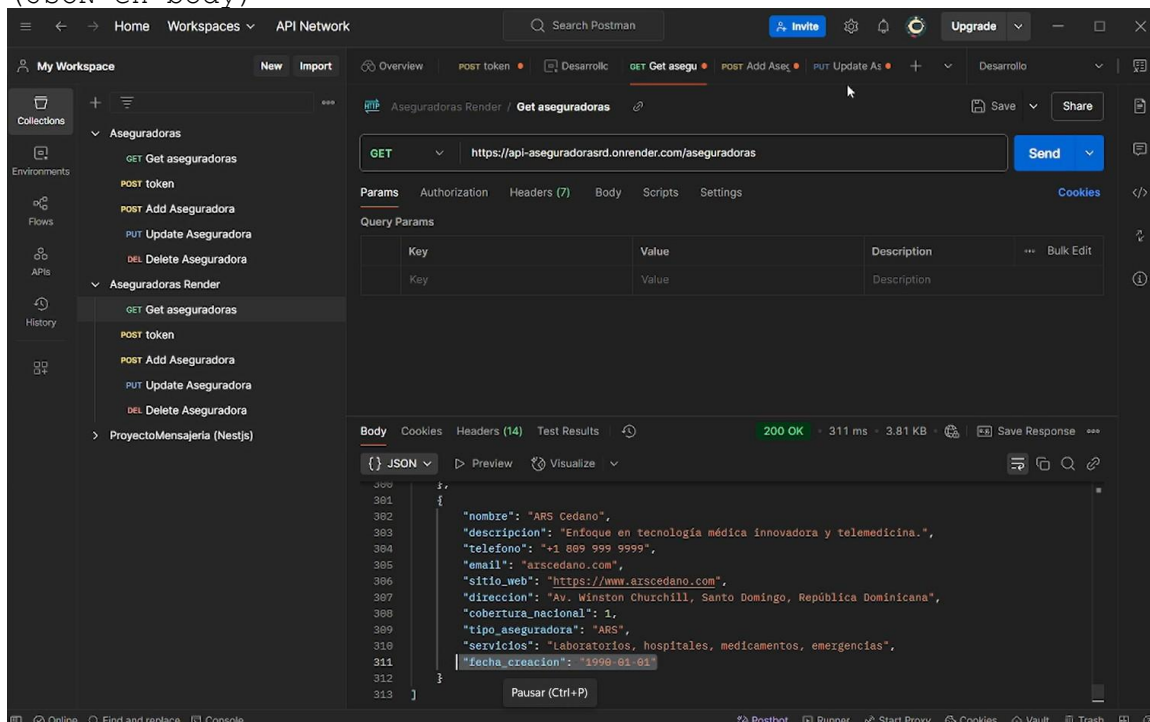
POST <https://api-aseguradorasrd.onrender.com/aseguradoras>
(con JSON en el body)



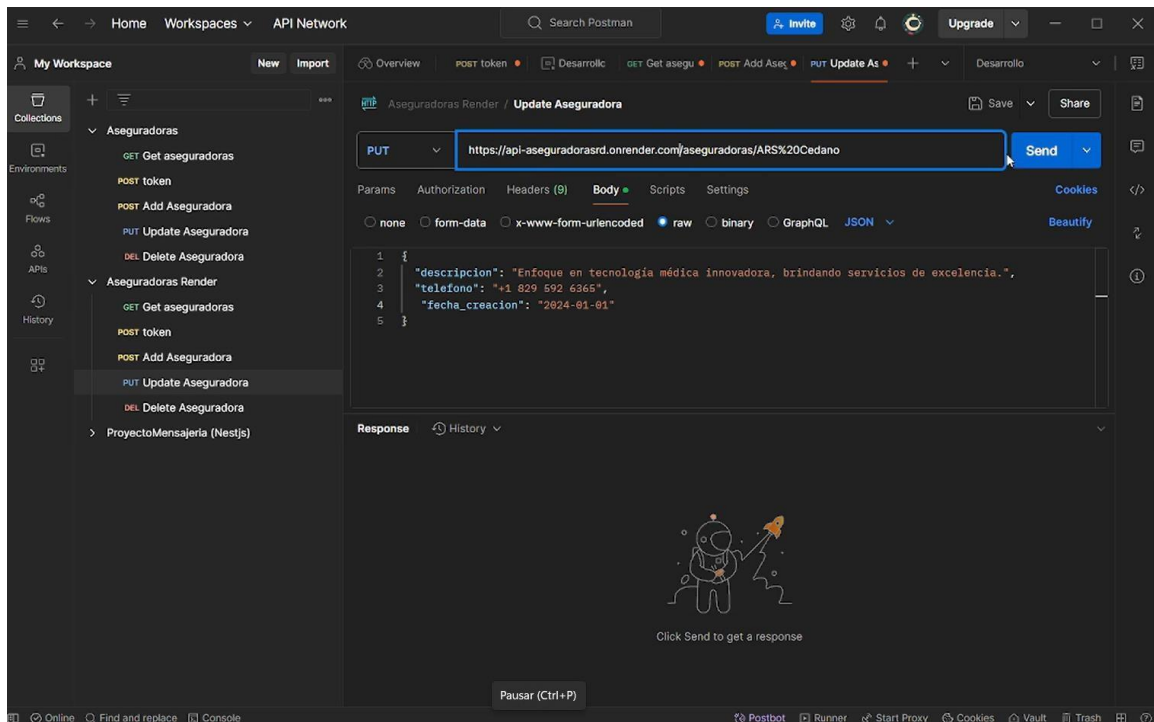


✓ Actualizar Aseguradora: PUT <https://api-aseguradorasrd.onrender.com/aseguradoras/ARS%20Cedano>

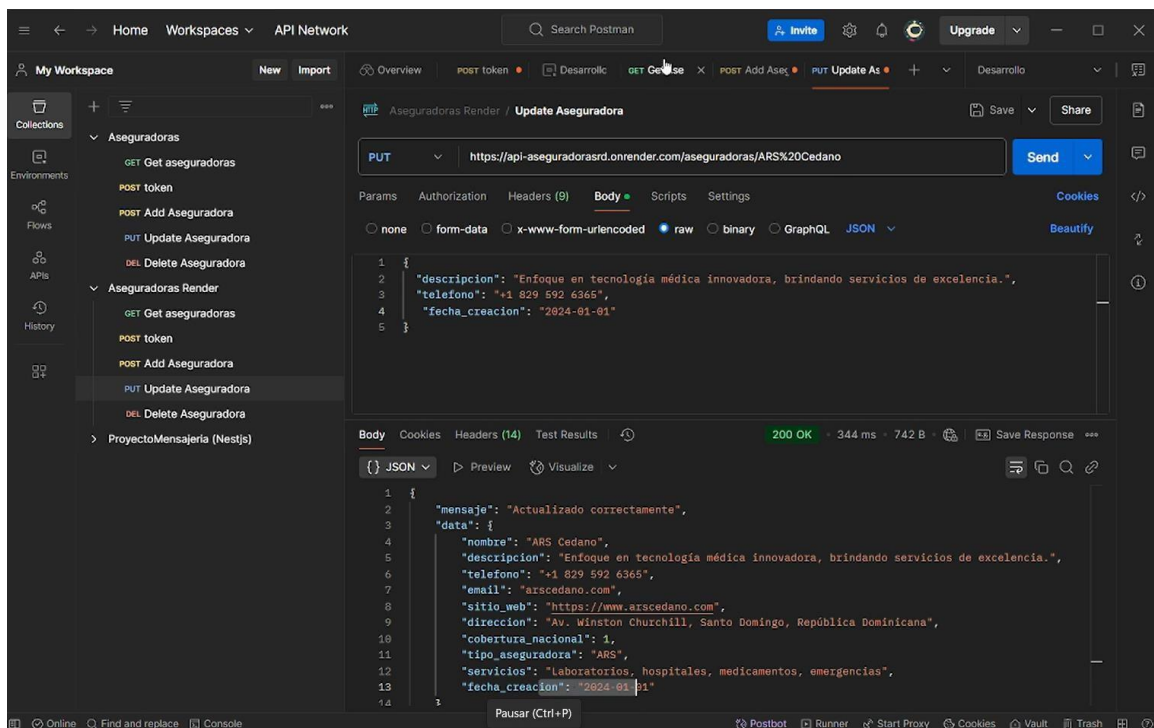
(JSON en body)



Primero en la url el formato es el siguiente: “`https://api-aseguradorasrd.onrender.com/aseguradoras/:nombre`” teniendo en cuenta que para poner los espacios usamos %20 en este ejemplo se usa una Asegurado que se hab\u00eda agregado a modo de ejemplo. Ponemos la informaci\u00f3n la cual queremos modificar:

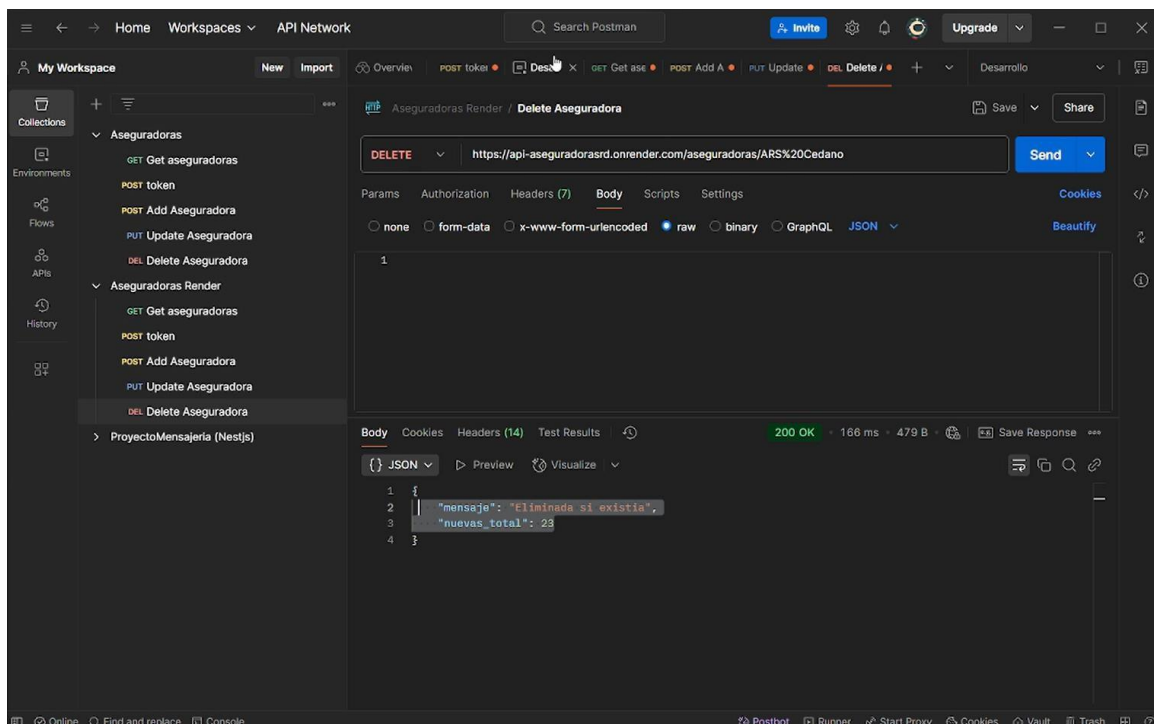


Y al darle enviar pues se actualiza la información suministrada.

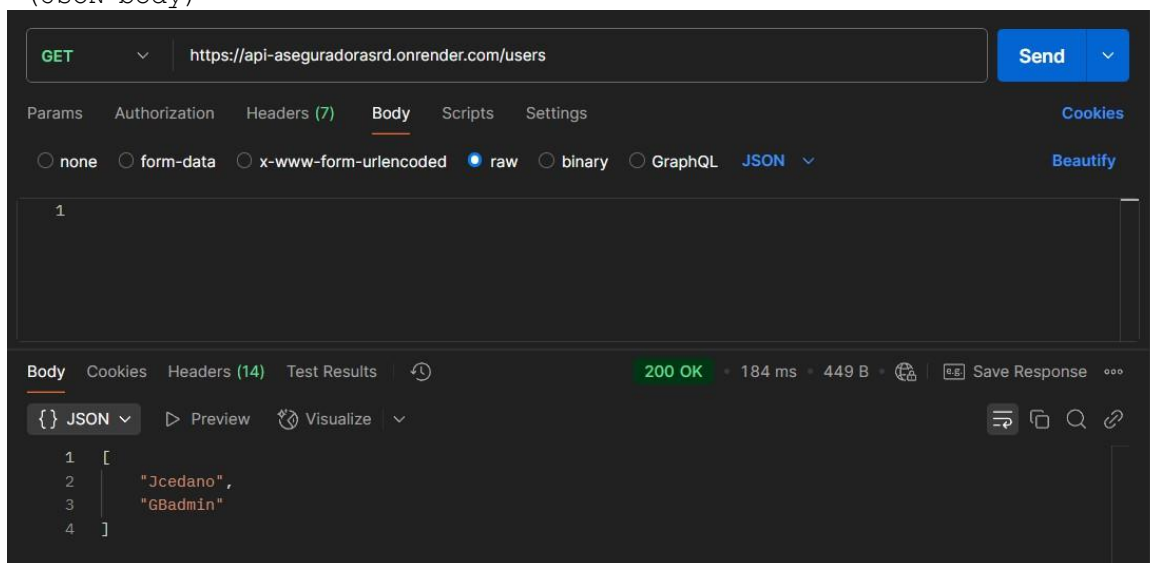


✓ **Eliminar Aseguradora: DEL** <https://api-aseguradorasrd.onrender.com/aseguradoras/ARS%20Cedano>

En dado caso que queramos eliminarla también podemos, simplemente especificando el nombre de la Aseguradora que queremos borrar:



✓ Obtener Usuarios: **GET** <https://api-aseguradorasrd.onrender.com/users>
(JSON body)

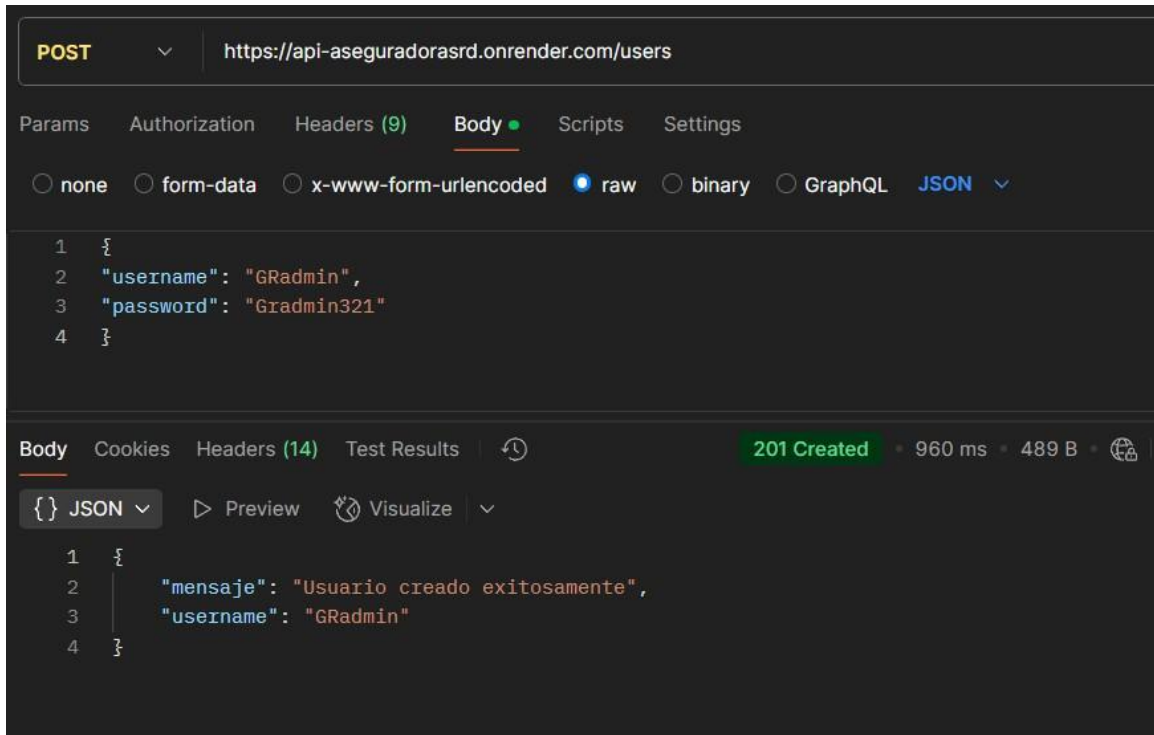


Una vez autenticado con uno de los usuarios y con su respectivo token en el header, puedes acceder a dos rutas protegidas mas, y una de estas es sensible la cual es la de los usuarios que tienen privilegio. Esta ruta es protegida y es por la razón de que se muestra los usuarios que están registrados actualmente, nótese que solo se muestra su username por razones de seguridad, la contraseña la cual tiene un hash no se muestra.

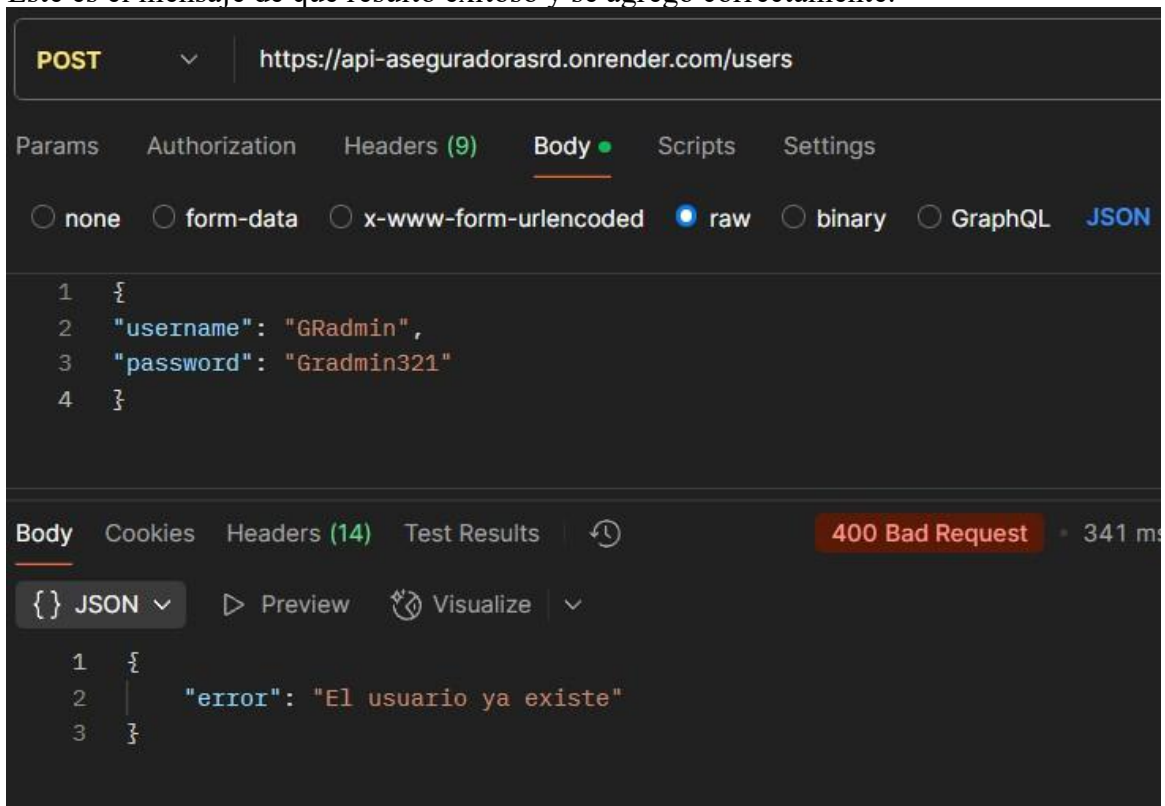
✓ Agregar Usuario: **POST** <https://api-aseguradorasrd.onrender.com/users>
(JSON body)

Esta ruta es protegida por una razón clara, se agregara un usuario con privilegios, por lo que requiere que tengas tu token para poder hacer dicha opción.

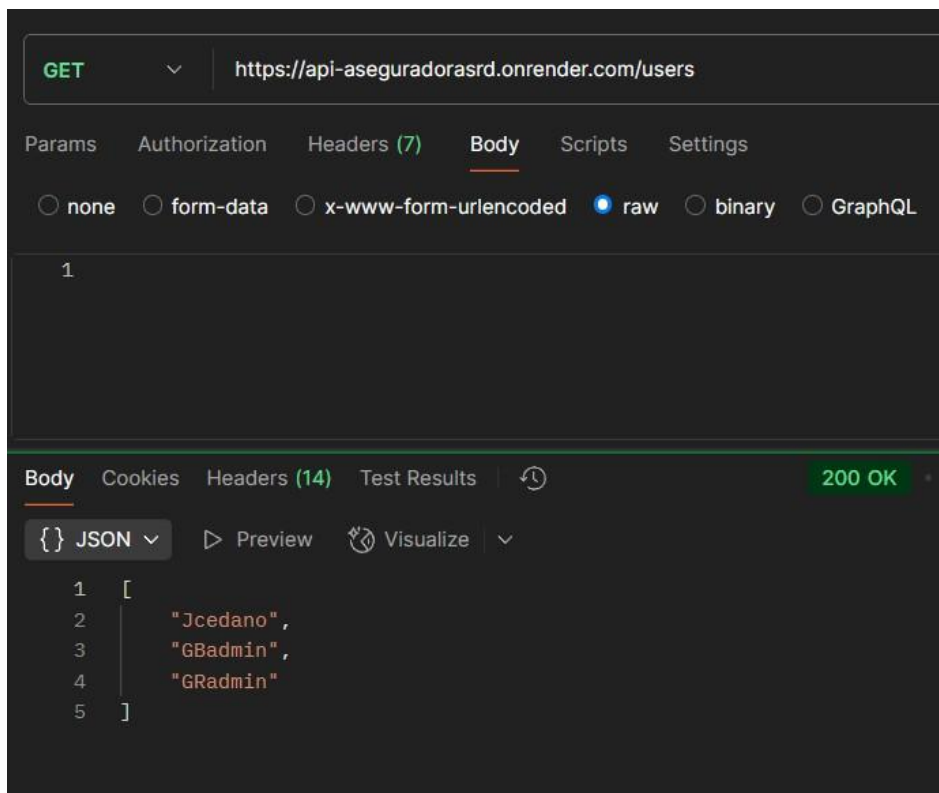
Funciona simple, una vez logeado, vas a la ruta y con el método POST vas a ingresar el username y password de tu nuevo usuario, este se guardara y agregara en users.json con su password hasheada e inmediatamente ese usuario estará disponible para poder utilizarse en la API.



Este es el mensaje de que resulto exitoso y se agrego correctamente.



Este mensaje si ingresas un usuario que ya existe. Ahora bien si queremos confirmar que nuestro user se agregó correctamente basta con hacer la petición GET a esta misma URL y nos lanzara lo siguiente:



Resultando así, en una petición exitosa.

Conclusión

La API Aseguradoras RD se ha desarrollado como una solución focalizada para la gestión de información del sector asegurador de salud en República Dominicana, combinando eficiencia técnica con usabilidad práctica. Su diseño responde a tres pilares fundamentales:

1. Especialización Sectorial

Centraliza y organiza datos clave de aseguradoras locales, ofreciendo un **catálogo digital estandarizado** con capacidad de búsqueda precisa. La autodocumentación integrada facilita su adopción por equipos técnicos y no técnicos.

2. Arquitectura Pragmática

Implementa un modelo **file-based** (JSON) que:

- Reduce dependencias externas
- Simplifica despliegues en entornos con limitaciones
- Mantiene un balance entre rendimiento y persistencia para cargas de trabajo moderadas

3. Seguridad Básica pero Robusta

Integra:

- Autenticación JWT
- Hashing de contraseñas con bcrypt
- Protección de endpoints críticos

Contribución Principal:

Esta API sirve como **punto** entre la necesidad de digitalización del sector y las restricciones técnicas comunes en entornos locales, demostrando que soluciones efectivas pueden construirse con tecnología accesible. Su estructura modular permite:

- Adaptarse a futuras integraciones con bases de datos formales
- Escalar hacia modelos de roles complejos
- Incorporar validaciones avanzadas según demanda

Lecciones Aprendidas:

El proyecto valida que el valor de una API no radica en su complejidad, sino en su capacidad para resolver problemas específicos. Queda como base para evolucionar hacia un sistema más granular, idealmente complementado con:

- Un dashboard de administración
- Documentación Swagger/OpenAPI
- Mecanismos de backup automatizados

En su estado actual, la API cumple su objetivo central: **automatizar la gestión de información aseguradora con un enfoque seguro, mantenible y alineado al contexto dominicano.**

